

Python Complete Reference Guide

Strings • Lists • Tuples • Dictionaries • Sets • Functions • File Handling •
Exceptions • Modules • OOP • Built-ins • Special Methods • math • re •
random

Every entry includes a short explanation, a runnable example, and its output.

Reference source: docs.python.org/3/

Table of Contents

1. String Methods (29 entries)
2. List Methods (19 entries)
3. Tuple Methods (10 entries)
4. Dictionary Methods (15 entries)
5. Set Methods (15 entries)
6. Functions (14 entries)
7. File Handling (12 entries)
8. Exception Handling (8 entries)
9. Modules & Packages (8 entries)
10. Classes & OOP (14 entries)
11. Built-in Functions (26 entries)
12. Special (Dunder) Methods (13 entries)
13. Math Module (11 entries)
14. Regular Expressions (re module) (9 entries)
15. Random Module (8 entries)
16. Other Useful Standard Modules (11 entries)

1. String Methods

Python strings are immutable sequences of characters. Below are the most commonly used string methods, each with a short working example.

str.upper()

Returns a copy of the string converted to uppercase.

EXAMPLE

```
s = "hello"
print(s.upper())
```

OUTPUT

```
HELLO
```

str.lower()

Returns a copy of the string converted to lowercase.

EXAMPLE

```
s = "HELLO"
print(s.lower())
```

OUTPUT

```
hello
```

str.title()

Returns the string with the first letter of each word capitalized.

EXAMPLE

```
s = "hello world"
print(s.title())
```

OUTPUT

```
Hello World
```

str.capitalize()

Capitalizes only the first character of the string.

EXAMPLE

```
s = "hello world"
print(s.capitalize())
```

OUTPUT

```
Hello world
```

str.swapcase()

Swaps uppercase characters to lowercase and vice versa.

EXAMPLE

```
s = "Hello"
print(s.swapcase())
```

OUTPUT

```
hELLO
```

str.strip()

Removes leading and trailing whitespace (or given characters).

EXAMPLE

```
s = "  hi  "
print(s.strip())
```

OUTPUT

```
hi
```

str.lstrip() / rstrip()

Removes whitespace from only the left or right side.

EXAMPLE

```
s = "  hi  "
print(s.lstrip(), s.rstrip())
```

OUTPUT

```
'hi' ' ' 'hi'
```

str.split(sep)

Splits a string into a list using the separator (default: whitespace).

EXAMPLE

```
s = "a,b,c"
print(s.split(","))
```

OUTPUT

```
['a', 'b', 'c']
```

str.rsplit(sep, maxsplit)

Splits from the right, limited by maxsplit.

EXAMPLE

```
s = "a,b,c"
print(s.rsplit(",", 1))
```

OUTPUT

```
['a,b', 'c']
```

str.splitlines()

Splits a string at line breaks.

EXAMPLE

```
s = "line1\nline2"
print(s.splitlines())
```

OUTPUT

```
['line1', 'line2']
```

str.join(iterable)

Joins elements of an iterable into a string using this string as separator.

EXAMPLE

```
s = "-"
print(s.join(["a", "b", "c"]))
```

OUTPUT

```
a-b-c
```

str.replace(old,new)

Replaces all occurrences of a substring with another.

EXAMPLE

```
s = "cat sat"
print(s.replace("at", "og"))
```

OUTPUT

```
cog sog
```

str.find(sub)

Returns the lowest index of the substring, or -1 if not found.

EXAMPLE

```
s = "hello"
print(s.find("ll"))
```

OUTPUT

2

str.rfind(sub)

Returns the highest index of the substring, or -1 if not found.

EXAMPLE

```
s = "abcabc"
print(s.rfind("a"))
```

OUTPUT

3

str.index(sub)

Like find(), but raises ValueError if not found.

EXAMPLE

```
s = "hello"
print(s.index("e"))
```

OUTPUT

1

str.count(sub)

Counts non-overlapping occurrences of a substring.

EXAMPLE

```
s = "banana"
print(s.count("an"))
```

OUTPUT

2

str.startswith(sub)

Checks if the string starts with the given substring.

EXAMPLE

```
s = "hello"
print(s.startswith("he"))
```

OUTPUT

True

str.endswith(sub)

Checks if the string ends with the given substring.

EXAMPLE

```
s = "hello"
print(s.endswith("lo"))
```

OUTPUT

True

str.format()

Formats a string using placeholders {}.

EXAMPLE

```
name = "Sam"
print("Hi {}".format(name))
```

OUTPUT

Hi Sam

f-strings

Embeds expressions directly inside string literals (Python 3.6+).

EXAMPLE

```
x = 5
print(f"x squared is {x**2}")
```

OUTPUT

```
x squared is 25
```

str.zfill(width)

Pads a numeric string with zeros on the left.

EXAMPLE

```
s = "42"
print(s.zfill(5))
```

OUTPUT

```
00042
```

str.center(width, ch)

Centers a string, padding with the given character.

EXAMPLE

```
s = "hi"
print(s.center(6, "*"))
```

OUTPUT

```
**hi**
```

str.ljust(width) / rjust(width)

Left/right-justifies a string within a given width.

EXAMPLE

```
s = "hi"
print(s.ljust(5, "-"), s.rjust(5, "-"))
```

OUTPUT

```
hi--- ---hi
```

str.isalpha() / isdigit() / isalnum()

Checks character composition: letters only, digits only, or alphanumeric.

EXAMPLE

```
print("abc".isalpha(), "123".isdigit(), "a1".isalnum())
```

OUTPUT

```
True True True
```

str.isspace() / isupper() / islower()

Checks for whitespace-only, all-uppercase, or all-lowercase strings.

EXAMPLE

```
print(" ".isspace(), "AB".isupper(), "ab".islower())
```

OUTPUT

```
True True True
```

str.encode()

Encodes the string into bytes using the given encoding.

EXAMPLE

```
s = "hi"
print(s.encode("utf-8"))
```

OUTPUT

```
b'hi'
```

str.partition(sep)

Splits the string at the first occurrence of sep into a 3-tuple.

EXAMPLE

```
s = "key=value"
print(s.partition("="))
```

OUTPUT

```
('key', '=', 'value')
```

len(str)

Returns the number of characters in the string.

EXAMPLE

```
s = "python"
print(len(s))
```

OUTPUT

```
6
```

str slicing

Extracts a substring using [start:stop:step] indexing.

EXAMPLE

```
s = "python"
print(s[1:4], s[::-1])
```

OUTPUT

```
yth nohtyp
```

2. List Methods

Lists are ordered, mutable sequences that can hold mixed data types.

list.append(x)

Adds an item to the end of the list.

EXAMPLE

```
a = [1,2]
a.append(3)
print(a)
```

OUTPUT

```
[1, 2, 3]
```

list.extend(iter)

Adds all elements of an iterable to the end of the list.

EXAMPLE

```
a = [1,2]
a.extend([3,4])
print(a)
```

OUTPUT

```
[1, 2, 3, 4]
```

list.insert(i,x)

Inserts an item x at position i.

EXAMPLE

```
a = [1,3]
a.insert(1,2)
print(a)
```

OUTPUT

```
[1, 2, 3]
```

list.remove(x)

Removes the first item with value x.

EXAMPLE

```
a = [1,2,3,2]
a.remove(2)
print(a)
```

OUTPUT

```
[1, 3, 2]
```

list.pop(i)

Removes and returns the item at index i (default: last).

EXAMPLE

```
a = [1,2,3]
print(a.pop())
print(a)
```

OUTPUT

```
3
[1, 2]
```

list.clear()

Removes all items from the list.

EXAMPLE

```
a = [1,2,3]
a.clear()
print(a)
```

OUTPUT

```
[]
```

list.index(x)

Returns the index of the first item equal to x.

EXAMPLE

```
a = [10,20,30]
print(a.index(20))
```

OUTPUT

```
1
```

list.count(x)

Counts occurrences of value x in the list.

EXAMPLE

```
a = [1,2,2,3]
print(a.count(2))
```

OUTPUT

```
2
```

list.sort()

Sorts the list in place (ascending by default).

EXAMPLE

```
a = [3,1,2]
a.sort()
print(a)
```

OUTPUT

```
[1, 2, 3]
```

list.sort(reverse=True)

Sorts the list in place in descending order.

EXAMPLE

```
a = [3,1,2]
a.sort(reverse=True)
print(a)
```

OUTPUT

```
[3, 2, 1]
```

list.reverse()

Reverses the list in place.

EXAMPLE

```
a = [1,2,3]
a.reverse()
print(a)
```

OUTPUT

```
[3, 2, 1]
```

list.copy()

Returns a shallow copy of the list.

EXAMPLE

```
a = [1,2]
b = a.copy()
```

```
print(b)
```

OUTPUT

```
[1, 2]
```

sorted(list)

Returns a new sorted list without modifying the original.

EXAMPLE

```
a = [3,1,2]
print(sorted(a))
```

OUTPUT

```
[1, 2, 3]
```

list comprehension

Creates a new list by applying an expression to each item.

EXAMPLE

```
a = [x*x for x in range(5)]
print(a)
```

OUTPUT

```
[0, 1, 4, 9, 16]
```

list slicing

Extracts a sublist using [start:stop:step].

EXAMPLE

```
a = [0,1,2,3,4]
print(a[1:4])
```

OUTPUT

```
[1, 2, 3]
```

len(list)

Returns the number of items in the list.

EXAMPLE

```
a = [1,2,3]
print(len(a))
```

OUTPUT

```
3
```

sum() / min() / max()

Aggregate functions for numeric lists.

EXAMPLE

```
a = [1,2,3]
print(sum(a), min(a), max(a))
```

OUTPUT

```
6 1 3
```

list + list / list * n

Concatenates lists or repeats a list n times.

EXAMPLE

```
a = [1,2]
print(a + [3,4], a*2)
```

OUTPUT

```
[1, 2, 3, 4] [1, 2, 1, 2]
```

in / not in

Checks membership of a value in the list.

EXAMPLE

```
a = [1,2,3]
print(2 in a)
```

OUTPUT

```
True
```

3. Tuple Methods

Tuples behave like lists but cannot be changed once created (immutable).

tuple creation

Tuples are ordered, immutable sequences created with parentheses.

EXAMPLE

```
t = (1, 2, 3)
print(t)
```

OUTPUT

```
(1, 2, 3)
```

tuple.count(x)

Counts occurrences of a value in the tuple.

EXAMPLE

```
t = (1, 2, 2, 3)
print(t.count(2))
```

OUTPUT

```
2
```

tuple.index(x)

Returns the index of the first occurrence of x.

EXAMPLE

```
t = (1, 2, 3)
print(t.index(3))
```

OUTPUT

```
2
```

tuple slicing

Extracts a portion of the tuple using [start:stop].

EXAMPLE

```
t = (0, 1, 2, 3)
print(t[1:3])
```

OUTPUT

```
(1, 2)
```

tuple unpacking

Assigns tuple elements to multiple variables at once.

EXAMPLE

```
t = (1, 2, 3)
a, b, c = t
print(a, b, c)
```

OUTPUT

```
1 2 3
```

len(tuple)

Returns the number of elements in the tuple.

EXAMPLE

```
t = (1, 2, 3)
print(len(t))
```

OUTPUT

```
3
```

tuple() constructor

Converts an iterable (like a list) into a tuple.

EXAMPLE

```
print(tuple([1,2,3]))
```

OUTPUT

```
(1, 2, 3)
```

tuple + tuple / * n

Concatenates tuples or repeats a tuple n times.

EXAMPLE

```
t = (1,2)
print(t + (3,4), t*2)
```

OUTPUT

```
(1, 2, 3, 4) (1, 2, 1, 2)
```

nested tuples

Tuples may contain other tuples or collections.

EXAMPLE

```
t = (1, (2,3), [4,5])
print(t[1][0])
```

OUTPUT

```
2
```

in / not in

Checks membership of a value in the tuple.

EXAMPLE

```
t = (1,2,3)
print(2 in t)
```

OUTPUT

```
True
```

4. Dictionary Methods

Dictionaries store data as key-value pairs and are mutable and unordered (insertion order is preserved since Python 3.7).

dict.get(k, default)

Returns the value for key k, or a default if the key is missing.

EXAMPLE

```
d = {"a":1}
print(d.get("b", 0))
```

OUTPUT

```
0
```

dict.keys()

Returns a view of all keys in the dictionary.

EXAMPLE

```
d = {"a":1, "b":2}
print(list(d.keys()))
```

OUTPUT

```
['a', 'b']
```

dict.values()

Returns a view of all values in the dictionary.

EXAMPLE

```
d = {"a":1, "b":2}
print(list(d.values()))
```

OUTPUT

```
[1, 2]
```

dict.items()

Returns a view of (key, value) pairs.

EXAMPLE

```
d = {"a":1, "b":2}
print(list(d.items()))
```

OUTPUT

```
[('a', 1), ('b', 2)]
```

dict.update(other)

Updates the dictionary with key/value pairs from another dict.

EXAMPLE

```
d = {"a":1}
d.update({"b":2})
print(d)
```

OUTPUT

```
{'a': 1, 'b': 2}
```

dict.pop(k)

Removes key k and returns its value.

EXAMPLE

```
d = {"a":1, "b":2}
print(d.pop("a"))
print(d)
```

OUTPUT

```
1
{'b': 2}
```

dict.popitem()

Removes and returns the last inserted (key, value) pair.

EXAMPLE

```
d = {"a":1,"b":2}
print(d.popitem())
```

OUTPUT

```
('b', 2)
```

dict.setdefault(k, default)

Returns value of k, inserting it with a default if missing.

EXAMPLE

```
d = {"a":1}
print(d.setdefault("b", 5))
print(d)
```

OUTPUT

```
5
{'a': 1, 'b': 5}
```

dict.clear()

Removes all items from the dictionary.

EXAMPLE

```
d = {"a":1}
d.clear()
print(d)
```

OUTPUT

```
{}
```

dict.copy()

Returns a shallow copy of the dictionary.

EXAMPLE

```
d = {"a":1}
e = d.copy()
print(e)
```

OUTPUT

```
{'a': 1}
```

dict comprehension

Builds a dictionary using an expression for each key/value pair.

EXAMPLE

```
d = {x: x*x for x in range(4)}
print(d)
```

OUTPUT

```
{0: 0, 1: 1, 2: 4, 3: 9}
```

del dict[k]

Deletes a key/value pair from the dictionary.

EXAMPLE

```
d = {"a":1,"b":2}
del d["a"]
print(d)
```

OUTPUT

```
{'b': 2}
```

in / not in

Checks whether a key exists in the dictionary.

EXAMPLE

```
d = {"a":1}
print("a" in d)
```

OUTPUT

```
True
```

dict(zip(keys,values))

Builds a dictionary by pairing two iterables together.

EXAMPLE

```
d = dict(zip(["a","b"],[1,2]))
print(d)
```

OUTPUT

```
{'a': 1, 'b': 2}
```

len(dict)

Returns the number of key/value pairs.

EXAMPLE

```
d = {"a":1,"b":2}
print(len(d))
```

OUTPUT

```
2
```

5. Set Methods

Sets are unordered collections of unique elements, useful for membership tests and mathematical set operations.

set.add(x)

Adds an element x to the set.

EXAMPLE

```
s = {1, 2}
s.add(3)
print(s)
```

OUTPUT

```
{1, 2, 3}
```

set.remove(x)

Removes x from the set; raises KeyError if not present.

EXAMPLE

```
s = {1, 2, 3}
s.remove(2)
print(s)
```

OUTPUT

```
{1, 3}
```

set.discard(x)

Removes x if present, does nothing otherwise (no error).

EXAMPLE

```
s = {1, 2}
s.discard(5)
print(s)
```

OUTPUT

```
{1, 2}
```

set.pop()

Removes and returns an arbitrary element from the set.

EXAMPLE

```
s = {1, 2, 3}
print(s.pop())
```

OUTPUT

```
1 (arbitrary)
```

set.union(other)

Returns a new set with elements from both sets.

EXAMPLE

```
a={1, 2}
b={2, 3}
print(a.union(b))
```

OUTPUT

```
{1, 2, 3}
```

set.intersection(other)

Returns a new set with elements common to both sets.

EXAMPLE

```
a={1, 2, 3}
```

```
b={2, 3, 4}
print(a.intersection(b))
```

OUTPUT

```
{2, 3}
```

set.difference(other)

Returns elements in the set that are not in the other.

EXAMPLE

```
a={1, 2, 3}
b={2, 3}
print(a.difference(b))
```

OUTPUT

```
{1}
```

set.symmetric_difference(other)

Returns elements in either set, but not both.

EXAMPLE

```
a={1, 2}
b={2, 3}
print(a.symmetric_difference(b))
```

OUTPUT

```
{1, 3}
```

set.issubset(other)

Checks if the set is a subset of another set.

EXAMPLE

```
a={1, 2}
b={1, 2, 3}
print(a.issubset(b))
```

OUTPUT

```
True
```

set.issuperset(other)

Checks if the set contains all elements of another set.

EXAMPLE

```
a={1, 2, 3}
b={1, 2}
print(a.issuperset(b))
```

OUTPUT

```
True
```

set.clear()

Removes all elements from the set.

EXAMPLE

```
s = {1, 2}
s.clear()
print(s)
```

OUTPUT

```
set()
```

set.copy()

Returns a shallow copy of the set.

EXAMPLE

```
s = {1, 2}
```

```
t = s.copy()
print(t)
```

OUTPUT

```
{1, 2}
```

frozenset()

Creates an immutable version of a set.

EXAMPLE

```
f = frozenset([1, 2, 3])
print(f)
```

OUTPUT

```
frozenset({1, 2, 3})
```

set comprehension

Builds a set using an expression for each element.

EXAMPLE

```
s = {x*x for x in range(4)}
print(s)
```

OUTPUT

```
{0, 1, 4, 9}
```

in / not in

Checks membership of a value in the set.

EXAMPLE

```
s = {1, 2, 3}
print(2 in s)
```

OUTPUT

```
True
```

6. Functions

Functions group reusable code. Python also supports anonymous functions, variable-length arguments, generators, and decorators.

def function

Defines a reusable block of code with the `def` keyword.

EXAMPLE

```
def greet(name):  
    return f"Hello, {name}"  
print(greet("Sam"))
```

OUTPUT

```
Hello, Sam
```

default arguments

Parameters can have default values used if no argument is passed.

EXAMPLE

```
def power(x, n=2):  
    return x**n  
print(power(3))
```

OUTPUT

```
9
```

****args***

Collects extra positional arguments into a tuple.

EXAMPLE

```
def total(*args):  
    return sum(args)  
print(total(1,2,3))
```

OUTPUT

```
6
```

*****kwargs***

Collects extra keyword arguments into a dictionary.

EXAMPLE

```
def info(**kwargs):  
    return kwargs  
print(info(name="Sam", age=20))
```

OUTPUT

```
{'name': 'Sam', 'age': 20}
```

lambda

Creates a small anonymous, one-expression function.

EXAMPLE

```
square = lambda x: x*x  
print(square(5))
```

OUTPUT

```
25
```

return

Sends a value back from the function to the caller.

EXAMPLE

```
def add(a,b):
```

```
        return a+b
    print(add(2,3))
```

OUTPUT

```
5
```

recursion

A function calling itself to solve a smaller sub-problem.

EXAMPLE

```
def fact(n):
    return 1 if n==0 else n*fact(n-1)
print(fact(5))
```

OUTPUT

```
120
```

map(func, iter)

Applies a function to every item of an iterable.

EXAMPLE

```
print(list(map(lambda x:x*2, [1,2,3])))
```

OUTPUT

```
[2, 4, 6]
```

filter(func, iter)

Filters items of an iterable for which the function returns True.

EXAMPLE

```
print(list(filter(lambda x:x%2==0, [1,2,3,4])))
```

OUTPUT

```
[2, 4]
```

reduce(func, iter)

Cumulatively applies a function to reduce an iterable to one value (from functools).

EXAMPLE

```
from functools import reduce
print(reduce(lambda a,b:a+b, [1,2,3,4]))
```

OUTPUT

```
10
```

generator (yield)

A function that yields values lazily one at a time.

EXAMPLE

```
def counter(n):
    for i in range(n):
        yield i
print(list(counter(3)))
```

OUTPUT

```
[0, 1, 2]
```

decorators

Wraps a function to add extra behaviour without changing its code.

EXAMPLE

```
def deco(f):
    def wrap():
        print("Before")
        f()
    return wrap
```

```
@deco
def hi():
    print("Hi")
hi()
```

OUTPUT

```
Before
Hi
```

****args unpacking on call***

Unpacks a list/tuple into positional arguments when calling.

EXAMPLE

```
def add3(a,b,c):
    return a+b+c
nums=[1,2,3]
print(add3(*nums))
```

OUTPUT

```
6
```

type hints

Optional annotations that document expected argument/return types.

EXAMPLE

```
def add(a: int, b: int) -> int:
    return a + b
print(add(2,3))
```

OUTPUT

```
5
```

7. File Handling

Python reads and writes files using `open()` together with modes, and the 'with' statement ensures files are closed automatically.

open(file, mode)

Opens a file and returns a file object. Common modes: r, w, a, r+, rb, wb.

EXAMPLE

```
f = open("test.txt", "w")
f.write("Hello")
f.close()
```

OUTPUT

```
(file created)
```

with open() as f

Context manager that auto-closes the file after the block.

EXAMPLE

```
with open("test.txt","w") as f:
    f.write("Hello World")
```

OUTPUT

```
(file written safely)
```

file.read()

Reads the entire content of the file as a string.

EXAMPLE

```
with open("test.txt") as f:
    print(f.read())
```

OUTPUT

```
Hello World
```

file.readline()

Reads a single line from the file.

EXAMPLE

```
with open("test.txt") as f:
    print(f.readline())
```

OUTPUT

```
Hello World
```

file.readlines()

Reads all lines and returns them as a list of strings.

EXAMPLE

```
with open("test.txt") as f:
    print(f.readlines())
```

OUTPUT

```
['Hello World']
```

file.write(str)

Writes a string to the file.

EXAMPLE

```
with open("test.txt","a") as f:
    f.write("\nMore text")
```

OUTPUT

```
(appended text)
```

file.writelines(list)

Writes a list of strings to the file.

EXAMPLE

```
with open("test.txt", "w") as f:
    f.writelines(["a\n", "b\n"])
```

OUTPUT

(file written)

file.seek(offset)

Moves the file pointer to a given byte position.

EXAMPLE

```
with open("test.txt") as f:
    f.seek(0)
    print(f.read(5))
```

OUTPUT

Hello

file.tell()

Returns the current position of the file pointer.

EXAMPLE

```
with open("test.txt") as f:
    f.read(5)
    print(f.tell())
```

OUTPUT

5

os.path.exists()

Checks whether a file or directory exists (os module).

EXAMPLE

```
import os
print(os.path.exists("test.txt"))
```

OUTPUT

True

os.remove(file)

Deletes a file from the filesystem.

EXAMPLE

```
import os
os.remove("test.txt")
```

OUTPUT

(file deleted)

csv.reader / writer

Reads and writes CSV files using the built-in csv module.

EXAMPLE

```
import csv
with open("a.csv", "w", newline="") as f:
    w=csv.writer(f)
    w.writerow(["a", "b"])
```

OUTPUT

(csv row written)

8. Exception Handling

Exceptions let a program gracefully respond to runtime errors using try/except/else/finally and custom exception classes.

try / except

Catches and handles an exception so the program doesn't crash.

EXAMPLE

```
try:
    1/0
except ZeroDivisionError:
    print("Cannot divide by zero")
```

OUTPUT

```
Cannot divide by zero
```

except Exception as e

Captures the exception object to inspect its message.

EXAMPLE

```
try:
    int("abc")
except ValueError as e:
    print(e)
```

OUTPUT

```
invalid literal for int() with base 10: 'abc'
```

multiple except blocks

Handles different exception types differently.

EXAMPLE

```
try:
    x = [1][5]
except IndexError:
    print("Index error")
except KeyError:
    print("Key error")
```

OUTPUT

```
Index error
```

else clause

Runs only if the try block completes without raising an exception.

EXAMPLE

```
try:
    x = 10
except Exception:
    print("error")
else:
    print("no error, x =", x)
```

OUTPUT

```
no error, x = 10
```

finally clause

Always executes, whether or not an exception occurred.

EXAMPLE

```
try:
    1/0
```

```
except ZeroDivisionError:
    print("error")
finally:
    print("done")
```

OUTPUT

```
error
done
```

raise

Manually triggers an exception.

EXAMPLE

```
def check(age):
    if age < 0:
        raise ValueError("Age can't be negative")
try:
    check(-1)
except ValueError as e:
    print(e)
```

OUTPUT

```
Age can't be negative
```

custom exceptions

Defines a new exception type by subclassing Exception.

EXAMPLE

```
class MyError(Exception):
    pass
try:
    raise MyError("custom problem")
except MyError as e:
    print(e)
```

OUTPUT

```
custom problem
```

assert

Raises AssertionError if a condition is False.

EXAMPLE

```
x = 5
assert x > 0, "x must be positive"
print("ok")
```

OUTPUT

```
ok
```

9. Modules & Packages

Modules let you organize and reuse code across files; packages group related modules together.

import module

Loads an entire module so its functions can be used.

EXAMPLE

```
import math
print(math.sqrt(16))
```

OUTPUT

```
4.0
```

from module import x

Imports specific names directly from a module.

EXAMPLE

```
from math import pi
print(pi)
```

OUTPUT

```
3.141592653589793
```

import module as alias

Imports a module under a shorter alias.

EXAMPLE

```
import numpy as np # example alias
import math as m
print(m.floor(3.7))
```

OUTPUT

```
3
```

dir(module)

Lists all names (functions/attributes) defined in a module.

EXAMPLE

```
import math
print("sqrt" in dir(math))
```

OUTPUT

```
True
```

creating your own module

Any .py file can be imported as a module in another file.

EXAMPLE

```
# mymodule.py
def hello():
    return "hi"

# main.py
import mymodule
print(mymodule.hello())
```

OUTPUT

```
hi
```

__name__ == "__main__"

Lets a file act as both a script and an importable module.

EXAMPLE

```
if __name__ == "__main__":  
    print("Run directly")
```

OUTPUT

```
Run directly
```

package (__init__.py)

A folder with `__init__.py` becomes an importable package of modules.

EXAMPLE

```
my_package/  
    __init__.py  
    module_a.py  
# import my_package.module_a
```

OUTPUT

```
(folder structure)
```

pip install

Installs third-party packages from PyPI via the command line.

EXAMPLE

```
pip install requests
```

OUTPUT

```
(installs package)
```

10. Classes & OOP

Object-Oriented Programming organizes code into classes and objects, supporting inheritance, encapsulation, and polymorphism.

class definition

Defines a blueprint for creating objects.

EXAMPLE

```
class Dog:
    def __init__(self, name):
        self.name = name
d = Dog("Rex")
print(d.name)
```

OUTPUT

Rex

__init__()

The constructor method, called automatically when an object is created.

EXAMPLE

```
class Point:
    def __init__(self,x,y):
        self.x, self.y = x, y
p = Point(1,2)
print(p.x, p.y)
```

OUTPUT

1 2

instance methods

Functions defined inside a class that operate on an instance (self).

EXAMPLE

```
class Circle:
    def __init__(self,r):
        self.r = r
    def area(self):
        return 3.14*self.r**2
print(Circle(2).area())
```

OUTPUT

12.56

class variables

Variables shared by all instances of a class.

EXAMPLE

```
class Dog:
    species = "Canine"
print(Dog.species)
```

OUTPUT

Canine

inheritance

Lets a class (child) reuse and extend another class (parent).

EXAMPLE

```
class Animal:
    def speak(self):
        return "..."
```

```
class Cat(Animal):
    def speak(self):
        return "Meow"
print(Cat().speak())
```

OUTPUT

Meow

super()

Calls a method from the parent class inside a subclass.

EXAMPLE

```
class A:
    def __init__(self):
        self.x = 1
class B(A):
    def __init__(self):
        super().__init__()
        self.y = 2
b=B()
print(b.x, b.y)
```

OUTPUT

1 2

method overriding

A subclass redefines a method inherited from the parent.

EXAMPLE

```
class A:
    def greet(self):
        return "Hi from A"
class B(A):
    def greet(self):
        return "Hi from B"
print(B().greet())
```

OUTPUT

Hi from B

encapsulation (_ / __)

Uses naming conventions to mark attributes as protected/private.

EXAMPLE

```
class Account:
    def __init__(self):
        self.__balance = 100
    def get_balance(self):
        return self.__balance
print(Account().get_balance())
```

OUTPUT

100

polymorphism

Different classes can implement the same method name differently.

EXAMPLE

```
class Cat:
    def sound(self): return "Meow"
class Dog:
    def sound(self): return "Woof"
for a in [Cat(), Dog()]:
    print(a.sound())
```

OUTPUT

```
Meow
Woof
```

@classmethod

A method bound to the class, not the instance, using cls.

EXAMPLE

```
class Dog:
    count = 0
    @classmethod
    def show(cls):
        return cls.count
print(Dog.show())
```

OUTPUT

```
0
```

@staticmethod

A method that doesn't use self or cls, grouped in the class for organization.

EXAMPLE

```
class MathUtil:
    @staticmethod
    def add(a,b):
        return a+b
print(MathUtil.add(2,3))
```

OUTPUT

```
5
```

@property

Turns a method into a read-only attribute-like accessor.

EXAMPLE

```
class Circle:
    def __init__(self,r):
        self.r = r
    @property
    def area(self):
        return 3.14*self.r**2
print(Circle(2).area)
```

OUTPUT

```
12.56
```

abstract classes (ABC)

Defines methods that subclasses are required to implement.

EXAMPLE

```
from abc import ABC, abstractmethod
class Shape(ABC):
    @abstractmethod
    def area(self): pass
class Sq(Shape):
    def area(self): return 4
print(Sq().area())
```

OUTPUT

```
4
```

isinstance() / issubclass()

Checks object type or class inheritance relationships.

EXAMPLE

```
class A: pass
```

```
class B(A): pass
print(isinstance(B(), A), issubclass(B, A))
```

OUTPUT

```
True True
```

11. Built-in Functions

Python provides many globally available functions without needing an import.

print()

Displays output to the console.

EXAMPLE

```
print("Hello", "World", sep="-")
```

OUTPUT

```
Hello-World
```

len()

Returns the number of items in a sequence/collection.

EXAMPLE

```
print(len("hello"))
```

OUTPUT

```
5
```

type()

Returns the type/class of an object.

EXAMPLE

```
print(type(5), type("hi"))
```

OUTPUT

```
<class 'int'> <class 'str'>
```

int() / float() / str()

Convert values between numeric and string types.

EXAMPLE

```
print(int("5"), float("2.5"), str(10))
```

OUTPUT

```
5 2.5 10
```

input()

Reads a line of text typed by the user.

EXAMPLE

```
name = input("Enter name: ")  
print(name)
```

OUTPUT

```
(waits for user input)
```

range()

Generates a sequence of numbers, often used in loops.

EXAMPLE

```
print(list(range(1,6)))
```

OUTPUT

```
[1, 2, 3, 4, 5]
```

enumerate()

Pairs each item of an iterable with its index.

EXAMPLE

```
for i,v in enumerate(["a","b"]):  
    print(i,v)
```

OUTPUT

```
0 a
1 b
```

zip()

Combines multiple iterables element-wise into tuples.

EXAMPLE

```
print(list(zip([1,2],["a","b"])))
```

OUTPUT

```
[(1, 'a'), (2, 'b')]
```

sorted()

Returns a new sorted list from any iterable.

EXAMPLE

```
print(sorted([3,1,2]))
```

OUTPUT

```
[1, 2, 3]
```

reversed()

Returns a reverse iterator over a sequence.

EXAMPLE

```
print(list(reversed([1,2,3])))
```

OUTPUT

```
[3, 2, 1]
```

sum() / min() / max()

Aggregate numeric functions over an iterable.

EXAMPLE

```
print(sum([1,2,3]), min([1,2,3]), max([1,2,3]))
```

OUTPUT

```
6 1 3
```

abs()

Returns the absolute (positive) value of a number.

EXAMPLE

```
print(abs(-7))
```

OUTPUT

```
7
```

round()

Rounds a number to a given number of decimal places.

EXAMPLE

```
print(round(3.14159, 2))
```

OUTPUT

```
3.14
```

pow()

Raises a number to a power (optionally modulo m).

EXAMPLE

```
print(pow(2,3))
```

OUTPUT

```
8
```

divmod()

Returns quotient and remainder as a tuple.

EXAMPLE

```
print(divmod(17,5))
```

OUTPUT

```
(3, 2)
```

all() / any()

Checks whether all/any elements of an iterable are truthy.

EXAMPLE

```
print(all([1,1,1]), any([0,0,1]))
```

OUTPUT

```
True True
```

isinstance()

Checks if an object is an instance of a given type.

EXAMPLE

```
print(isinstance(5, int))
```

OUTPUT

```
True
```

id()

Returns the unique memory identity of an object.

EXAMPLE

```
x = 5  
print(id(x) == id(x))
```

OUTPUT

```
True
```

eval()

Evaluates a string as a Python expression.

EXAMPLE

```
print(eval("2+3*4"))
```

OUTPUT

```
14
```

map() / filter()

Apply a function across an iterable, or filter its items.

EXAMPLE

```
print(list(map(str, [1,2,3])))
```

OUTPUT

```
['1', '2', '3']
```

list() / tuple() / set() / dict()

Type constructors that convert between collection types.

EXAMPLE

```
print(list((1,2,3)))
```

OUTPUT

```
[1, 2, 3]
```

open()

Opens a file and returns a file object (see File Handling section).

EXAMPLE

```
f = open("t.txt", "w")  
f.close()
```

OUTPUT

```
(file object)
```

help()

Displays documentation for an object, module, or function.

EXAMPLE

```
help(str.upper) # prints docstring
```

OUTPUT

```
(prints docs)
```

format()

Formats a value according to a format spec.

EXAMPLE

```
print(format(3.14159, ".2f"))
```

OUTPUT

```
3.14
```

chr()* / *ord()

Convert between a Unicode code point and its character.

EXAMPLE

```
print(chr(65), ord("A"))
```

OUTPUT

```
A 65
```

hex()* / *oct()* / *bin()

Convert an integer to hexadecimal, octal, or binary string.

EXAMPLE

```
print(hex(255), oct(8), bin(5))
```

OUTPUT

```
0xff 0o10 0b101
```

12. Special (Dunder) Methods

Double-underscore 'magic' methods let custom classes work with built-in operators and functions like +, ==, len(), and for-loops.

`__init__(self,...)`

Constructor, called when a new object is created.

EXAMPLE

```
class P:
    def __init__(self,x):
        self.x=x
print(P(5).x)
```

OUTPUT

5

`__str__(self)`

Defines the human-readable string shown by print()/str().

EXAMPLE

```
class P:
    def __str__(self):
        return "Point!"
print(P())
```

OUTPUT

Point!

`__repr__(self)`

Defines the official/debug string representation of an object.

EXAMPLE

```
class P:
    def __repr__(self):
        return "P()"
p=P()
print(repr(p))
```

OUTPUT

P()

`__len__(self)`

Allows len() to be called on your custom object.

EXAMPLE

```
class Bag:
    def __len__(self):
        return 3
print(len(Bag()))
```

OUTPUT

3

`__add__(self,other)`

Defines behaviour for the + operator on custom objects.

EXAMPLE

```
class V:
    def __init__(self,x): self.x=x
    def __add__(self,o): return V(self.x+o.x)
print((V(2)+V(3)).x)
```

OUTPUT

`__eq__(self,other)`

Defines behaviour for the == comparison operator.

EXAMPLE

```
class P:
    def __init__(self,x): self.x=x
    def __eq__(self,o): return self.x==o.x
print(P(1)==P(1))
```

OUTPUT

True

`__lt__(self,other)`

Defines behaviour for the < comparison operator (enables sorting).

EXAMPLE

```
class P:
    def __init__(self,x): self.x=x
    def __lt__(self,o): return self.x<o.x
print(P(1)<P(2))
```

OUTPUT

True

`__getitem__(self,i)`

Allows indexing (obj[i]) on a custom object.

EXAMPLE

```
class Seq:
    def __getitem__(self,i): return i*2
print(Seq()[3])
```

OUTPUT

6

`__setitem__(self,i,v)`

Allows item assignment (obj[i]=v) on a custom object.

EXAMPLE

```
class Box:
    def __init__(self): self.d={}
    def __setitem__(self,k,v): self.d[k]=v
b=Box()
b["a"]=1
print(b.d)
```

OUTPUT

{'a': 1}

`__call__(self,...)`

Makes an instance callable like a function, e.g. obj().

EXAMPLE

```
class Adder:
    def __call__(self,a,b): return a+b
add=Adder()
print(add(2,3))
```

OUTPUT

5

`__iter__ / __next__`

Makes a custom object iterable, usable in for-loops.

EXAMPLE

```
class Counter:
    def __init__(self,n): self.n,self.i=n,0
    def __iter__(self): return self
    def __next__(self):
        if self.i>=self.n: raise StopIteration
        self.i+=1
        return self.i
print(list(Counter(3)))
```

OUTPUT

```
[1, 2, 3]
```

__contains__(self,x)

Defines behaviour for the 'in' operator on custom objects.

EXAMPLE

```
class Box:
    def __contains__(self,x): return x==5
print(5 in Box())
```

OUTPUT

```
True
```

__del__(self)

Called when an object is about to be destroyed (destructor).

EXAMPLE

```
class Temp:
    def __del__(self): print("Deleted")
t=Temp()
del t
```

OUTPUT

```
Deleted
```

13. Math Module

The math module provides access to standard mathematical functions and constants.

math.sqrt(x)

Returns the square root of x.

EXAMPLE

```
import math
print(math.sqrt(25))
```

OUTPUT

```
5.0
```

math.pow(x,y)

Returns x raised to the power y (always float).

EXAMPLE

```
import math
print(math.pow(2, 3))
```

OUTPUT

```
8.0
```

math.ceil(x) / floor(x)

Rounds up / down to the nearest integer.

EXAMPLE

```
import math
print(math.ceil(4.2), math.floor(4.8))
```

OUTPUT

```
5 4
```

math.factorial(n)

Returns n! (the factorial of n).

EXAMPLE

```
import math
print(math.factorial(5))
```

OUTPUT

```
120
```

math.gcd(a,b)

Returns the greatest common divisor of a and b.

EXAMPLE

```
import math
print(math.gcd(12, 18))
```

OUTPUT

```
6
```

math.pi / math.e

Predefined mathematical constants Pi and Euler's number.

EXAMPLE

```
import math
print(math.pi, math.e)
```

OUTPUT

```
3.14159... 2.71828...
```

math.log(x, base)

Returns the logarithm of x to the given base (default e).

EXAMPLE

```
import math
print(math.log(100,10))
```

OUTPUT

2.0

math.sin/cos/tan(x)

Trigonometric functions (input in radians).

EXAMPLE

```
import math
print(round(math.sin(math.pi/2),2))
```

OUTPUT

1.0

math.degrees()/radians()

Converts between radians and degrees.

EXAMPLE

```
import math
print(math.degrees(math.pi))
```

OUTPUT

180.0

math.isnan()/isinf()

Checks if a value is 'Not a Number' or infinite.

EXAMPLE

```
import math
print(math.isnan(float("nan")))
```

OUTPUT

True

math.hypot(x,y)

Returns the Euclidean distance $\sqrt{x^2+y^2}$.

EXAMPLE

```
import math
print(math.hypot(3,4))
```

OUTPUT

5.0

14. Regular Expressions (re module)

The re module provides pattern-matching operations for searching, splitting, and replacing text.

re.match(pattern,str)

Checks for a match only at the beginning of the string.

EXAMPLE

```
import re
print(bool(re.match(r"Hi", "Hi there")))
```

OUTPUT

```
True
```

re.search(pattern,str)

Searches the whole string for the first match.

EXAMPLE

```
import re
print(bool(re.search(r"there", "Hi there")))
```

OUTPUT

```
True
```

re.findall(pattern,str)

Returns a list of all non-overlapping matches.

EXAMPLE

```
import re
print(re.findall(r"\d+", "a1 b22 c333"))
```

OUTPUT

```
['1', '22', '333']
```

re.finditer(pattern,str)

Returns an iterator of match objects for all matches.

EXAMPLE

```
import re
for m in re.finditer(r"\d+", "a1 b22"):
    print(m.group())
```

OUTPUT

```
1
22
```

re.sub(pattern,repl,str)

Replaces all matches of a pattern with a replacement string.

EXAMPLE

```
import re
print(re.sub(r"\d+", "#", "a1 b22"))
```

OUTPUT

```
a# b#
```

re.split(pattern,str)

Splits a string wherever the pattern matches.

EXAMPLE

```
import re
print(re.split(r"\s+", "a b c"))
```

OUTPUT

```
['a', 'b', 'c']
```

re.compile(pattern)

Pre-compiles a pattern for repeated, efficient matching.

EXAMPLE

```
import re
p = re.compile(r"\d+")
print(p.findall("a1 b2"))
```

OUTPUT

```
['1', '2']
```

groups ()

Captures parts of a match for later retrieval via group().

EXAMPLE

```
import re
m = re.match(r"(\w+)@(\w+)", "user@host")
print(m.group(1), m.group(2))
```

OUTPUT

```
user host
```

common patterns

\d digit, \w word char, \s whitespace, + one or more, * zero or more, ^ start, \$ end.

EXAMPLE

```
import re
print(re.findall(r"^\w+", "Hello World"))
```

OUTPUT

```
['Hello']
```

15. Random Module

The random module generates pseudo-random numbers and performs random selections/shuffles.

random.random()

Returns a random float between 0.0 and 1.0.

EXAMPLE

```
import random
print(0 <= random.random() < 1)
```

OUTPUT

True

random.randint(a,b)

Returns a random integer N such that $a \leq N \leq b$.

EXAMPLE

```
import random
print(1 <= random.randint(1,10) <= 10)
```

OUTPUT

True

random.uniform(a,b)

Returns a random float N such that $a \leq N \leq b$.

EXAMPLE

```
import random
print(1 <= random.uniform(1,5) <= 5)
```

OUTPUT

True

random.choice(seq)

Returns a single random element from a non-empty sequence.

EXAMPLE

```
import random
print(random.choice(["a","b","c"]) in ["a","b","c"])
```

OUTPUT

True

random.choices(seq,k=n)

Returns a list of n random elements, with replacement.

EXAMPLE

```
import random
print(len(random.choices([1,2,3], k=5)))
```

OUTPUT

5

random.sample(seq,k)

Returns k unique random elements (no repeats).

EXAMPLE

```
import random
print(len(random.sample(range(10),3)))
```

OUTPUT

3

random.shuffle(list)

Shuffles a list in place, randomizing its order.

EXAMPLE

```
import random
a=[1,2,3,4]
random.shuffle(a)
print(sorted(a))
```

OUTPUT

```
[1, 2, 3, 4]
```

random.seed(n)

Sets the random seed so results are reproducible.

EXAMPLE

```
import random
random.seed(1)
print(random.randint(1,10))
```

OUTPUT

```
(fixed value)
```

16. Other Useful Standard Modules

A few more commonly used modules from the Python standard library.

datetime.now()

Returns the current local date and time.

EXAMPLE

```
from datetime import datetime
print(datetime.now().year > 2000)
```

OUTPUT

```
True
```

time.sleep(sec)

Pauses program execution for a given number of seconds.

EXAMPLE

```
import time
time.sleep(0.1)
print("done")
```

OUTPUT

```
done
```

os.getcwd()

Returns the current working directory.

EXAMPLE

```
import os
print(type(os.getcwd()))
```

OUTPUT

```
<class 'str'>
```

os.listdir(path)

Lists all files/folders in a directory.

EXAMPLE

```
import os
print(type(os.listdir(".")))
```

OUTPUT

```
<class 'list'>
```

sys.argv

List of command-line arguments passed to a script.

EXAMPLE

```
import sys
print(type(sys.argv))
```

OUTPUT

```
<class 'list'>
```

json.dumps() / loads()

Converts Python objects to/from JSON strings.

EXAMPLE

```
import json
s = json.dumps({"a":1})
print(json.loads(s))
```

OUTPUT

```
{'a': 1}
```

collections.Counter

Counts hashable items in an iterable.

EXAMPLE

```
from collections import Counter
print(Counter("banana"))
```

OUTPUT

```
Counter({'a': 3, 'n': 2, 'b': 1})
```

collections.defaultdict

A dict subclass that supplies a default value for missing keys.

EXAMPLE

```
from collections import defaultdict
d = defaultdict(int)
d["x"] += 1
print(d["x"])
```

OUTPUT

```
1
```

itertools.permutations

Generates all possible orderings of an iterable.

EXAMPLE

```
from itertools import permutations
print(list(permutations([1,2])))
```

OUTPUT

```
[(1, 2), (2, 1)]
```

itertools.combinations

Generates all possible r-length combinations.

EXAMPLE

```
from itertools import combinations
print(list(combinations([1,2,3],2)))
```

OUTPUT

```
[(1, 2), (1, 3), (2, 3)]
```

functools.lru_cache

Caches function results to speed up repeated calls.

EXAMPLE

```
from functools import lru_cache
@lru_cache
def f(n): return n*n
print(f(5))
```

OUTPUT

```
25
```